

SPECIAL ISSUE PAPER

Comparing pre-commit reviews and post-commit reviews using process simulation

Tobias Baum¹  | Fabian Kortum¹ | Kurt Schneider¹ | Arthur Brack² | Jens Schauder³

¹FG Software Engineering, Leibniz Universität Hannover, Hannover, Germany

²SET GmbH, Hannover, Germany

³T-Systems on site services GmbH, Wolfsburg, Germany

Correspondence

Tobias Baum, FG Software Engineering, Leibniz Universität Hannover, Welfengarten 1, 30167 Hannover, Germany.

Email: tobias.baum@inf.uni-hannover.de

Abstract

Code review in practice is often performed change-based, ie, using the code changes belonging to a task to determine which code to review. Previous studies found that 2 variations of this process are used in industry: pre-commit review and post-commit review. The choice between these has implications not only for practitioners deciding on a code review process to use but also for the development of review tools and for experimentation with review processes. In some situations, a specific variant is clearly preferable due to the nature of the development process or team. In other situations, there are conflicting opinions. So we asked: Are there practically relevant performance differences between pre and post-commit reviews? How are these differences influenced by contextual factors? To assess these questions, we designed a parametric discrete event simulation model of certain agile development processes. We validated this model with practitioner's feedback and in part also with empirical data from industry. Our analysis indicates that the best choice does depend on the context but also that there are many situations with no practically relevant difference between both choices. We identified the main influencing factors and underlying effects and condensed our findings into heuristic rules.

KEYWORDS

agile software development, code review, discrete event simulation, post-commit review, pre-commit review

1 | INTRODUCTION

Code review, in particular in the form of formal inspection,^{1,2} has been recognized as an effective quality assurance technique for decades. In recent years, "continuous differential code review,"³ a form of code review that is based on the frequent review of small code changes and firmly embedded into the development process, has become the dominant style of code review, especially in open source and agile development.^{4,5} Our long-term goal is to improve the support for this style of review in the scope of small and medium software enterprises, which motivates our interest in commonalities and differences between industrial review processes. One of these differences is the "visibility" of the code under review. In some cases, the review is performed before the changes become visible for other developers who are working on the source code repository trunk/master. This form is often called "pre-commit review" or "review-then-commit". In most other cases, the review is performed after integrating the changes

into the trunk ("post-commit review"). Pre-commit review has recently gained popularity,⁴ especially in the form of "pull requests".⁶ This sparks discussions in development teams on the preferable way to perform reviews. To better understand the variations in review processes in practice, we performed in-depth interviews in 19 software companies, ranging from small start-ups and standard software producers to large in-house IT departments.^{5,7} In these interviews, the software development professionals named several reasons why they believe their method of choice is more efficient:

- Pre-commit reviews find defects before they impede other developers.
- Pre-commit reviews might extend the cycle time of user stories, increasing the "work in progress" and consequently increasing task switch overhead.
- Post-commit reviews support the early and often integration of changes, better suiting the mindset behind continuous integration and possibly reducing the risk of conflicts.

Our hypothesis is that the best choice depends on the context*, and that some of these effects have a larger influence than others. Consequently, our research questions are *Are there practically relevant performance differences between pre-commit and post-commit reviews? How are these differences influenced by contextual factors?* As these questions are still quite vague, they will be further detailed in section 3. A second study goal is to gain a better understanding of the effects responsible for performance differences and to use this knowledge to frame a set of simple, practically usable heuristics that can help teams to decide which variant is adequate for their situation.

We chose a simulation-based approach to assess these research questions, instead of a controlled experiment or a number of case studies. Performing a simulation allows us to study a vast number of different situations in a holistic way and with significantly less effort than a large scale experiment.

In summary, we aim to contribute the following:

- An analysis of the effects and contextual factors that influence whether pre-commit or post-commit reviews lead to better results in efficiency, quality, or cycle time.
- A set of heuristics that can help practitioners with the choice between pre and post-commit reviews.
- An open source implementation of a parametric discrete event simulation model of agile development processes with different styles of code review that can be used for further validation of our results as well as for further independent research.⁹

The current article is an extended version of our work presented at ICSSP 2016.¹⁰ It provides an extended discussion, further empirical foundations for the modeling decisions, and more details on the model.

2 | RELATED WORK

Our research questions and the design of large parts of our model are based on recent studies of code review processes in practice. Rigby and Bird⁴ give a consolidated overview of several studies on code review practices in open source and large industrial software development. Our own interviews⁵ support their results and provide us with further information on process variants and the arguments behind them. In Rigby's dissertation,¹¹ he studies OSS peer reviews through a multi-case study methodology. His thesis also contains a comparison of pre and post-commit reviews. He notices a shorter review interval and slightly fewer issues found with post-commit reviews and hypothesizes that these are due to anomalies in the data caused by differing contexts.

We are not aware of any simulation studies explicitly targeting "modern" change-based code review, but many studies used simulation to examine classic forms of inspection and review. Among these are system dynamics models, like the ones by Madachy¹² and Tvedt and Collofello,¹³ as well as discrete event models, for example by Neu et al¹⁴ and Rus, Halling and Biffi.¹⁵ The requirements inspection process has

been studied by Münch and Armbrust,¹⁶ as well as Wakeland, Martin, and Raffo.¹⁷

Simulations have been used to assess other questions in the context of agile software development. Turnu et al¹⁸ used a system dynamics model to investigate the effect of test-driven development in open source development, and Melis et al¹⁹ built a model to analyze the effects of test-driven development and pair programming. Anderson et al²⁰ used simulation to compare Scrum, Kanban, and a classic process for a team at Microsoft. A comprehensive overview of further software process simulation studies can be gained from the systematic literature reviews done by Zhang, Kitchenham, and Pfahl²¹ and Ali, Petersen, and Wohlin.²²

The choice between pre-commit and post-commit reviews is one of the many choices that determines the review process,⁵ and code reviews itself are part of the general software development process. There is evidence that the ability to adapt the software development process to contextual/situational factors is beneficial for a company's success, eg, shown by Clarke et al²³ and MacCormack and Verganti.²⁴ There exist ambitious research efforts to collect and systematize the relevant situational factors and their influence on process choices. The situational factors reference framework published by Clarke and O'Connor⁸ is the most comprehensive collection of situational factors to date. In their MoSAIC framework,²⁵ Jeners et al connect these situational factors to process repositories, ie, collections of process models.

3 | METHODOLOGY

From a very high-level view, our "in silico" study consists of 2 iterations of the scientific cycle of induction and deduction: We start with mostly qualitative empirical observations, create a simulation model (ie, an executable theory) based on these observations, use this model to deduce quantitative data, which we then use to validate the model but also as the starting point of another cycle, inducing heuristics and checking them against the simulation results. A closer look reveals that each cycle, in fact, consists of several iterations of refinement and validation.

3.1 | Specific research questions

The term "performance difference" from our general research question is too vague to be used directly. Following the goal question metric paradigm, we first split it into 3 questions corresponding to the classic target dimensions of software projects: quality, cost/efficiency, and time to market. The chosen metric for quality is "number of issues found by customers per story point"[†]. For efficiency, we use "number of story points completed", because the actual effort (ie, number of developers times simulated duration) is fixed for a given situation. For time to

*The "context" in this case is everything outside the narrow realm of pre vs post-commit reviews. It is therefore different to the definition of context used for example in the situational factor reference framework.⁸ The relation between both models will be discussed in section 7.3.

[†]We first used "number of issues found by customers"; but this is highly correlated with the amount of work finished.

market, we use the “mean user story cycle time”, ie, the average time from the start of a story to its delivery to the customer. We checked the adequacy of these measures in a questionnaire answered by 12

industrial software developers (a convenience sample taken from the local Java User Group). The respondents also gave bounds for the differences that they regard as practically relevant in the context of our

TABLE 1 Main model parameters (1/2)

Name/topic	Description	Sampling range (section 5.2)	Example value (section 5.1)
Issue injection rate	The mode and width of a triangular distribution giving the implementation skills of the developers, measured in issues injected per implementation task hour.	0.0 - 1.5 i./h 0.0 - 0.4 i./h	0.26 i./h [†] 0.1 i./h
Review skill	The mode and width of a triangular distribution giving the code reviewing skills of the developers, measured as the probability of detecting an issue in review.	0.0 - 1.0 0.0 - 0.3	0.45 0.05
Global blocker issue risk	The probability that a global blocker issue is injected while implementing a task or fix.	0.0 - 0.1	0.001
Global blocker issue suspend time	Mode of a triangular distribution taken for sampling the time for interruptions through global blocker issues.	0.01 - 3.0 h	0.15 h
Conflict probability	The probability that a conflict occurs between a task and another specific task that was committed between the first task's update and now.	0.0 - 0.1	0.012
Conflict resolution time	Mode of a triangular distribution taken for sampling the time for resolving a conflict on commit.	0.1 - 3.0 h	0.3 h
Implementation task duration	Mode and difference between mode and mean for the distribution from which a story task's duration (without overhead) is sampled. A log-normal distribution provided the best fit to the empirical data, a shifted exponential distribution was used as a second alternative (this applies to all durations).	0.1 - 2.0 h 0.1 - 28.0 h	0.2 h [†] 9.5 h [†]
Review duration	Mode and difference between mode and mean for the distribution from which the time needed for a single review round is sampled.	0.02 - 1.0 h 0.1 - 6.0 h	0.0323 h [†] 1.625 h [†]
Review remark fix duration	Mode and difference between mode and mean for the distribution from which the time needed to fix a single review remark is sampled.	0.01 - 0.25 h 0.01 - 4.0 h	0.02 h 0.7 h
Issue fix task overhead	Mode and difference between mode and mean for the distribution from which the overhead analysis time for fixing an issue as an issue fix task (in addition to the time needed to fix it as review remark) is sampled.	0.01 - 2.0 h 0.1 - 28.0 h	0.0132 h [†] 5.5 h [†]
Issue assessment duration	Mode and difference between mode and mean for the distribution from which the time needed for issue assessment is sampled.	0.01 - 1.0 h 0.1 - 4.0 h	0.1 h 0.35 h
Internal issue share	Share of issues that cannot be found by customers.	0.0 - 1.0	0.5
Issue activation time - Developer	Shift and mean for a shifted exponential distribution giving the time between committing an issue and its surfacing to a developer.	0.0 - 8.0 h 4 - 4000 h	0.5 h 2000 h
Issue activation time - Customer	Shift and mean for a shifted exponential distribution giving the time between “delivering” an issue and its surfacing to a customer.	4.0 h 4 - 4000 h	4.0 h 1000 h [†]
Planning duration	Mean of the distribution from which the time needed for planning a story is sampled.	0.1 - 30.0 h	4.0 h
Developer count	Number of developers working in the team/same part of the software.	3 - 29	12 [†]
Task switch overhead	Time it takes for a developer to re-familiarize himself with a topic after working on another topic for (1) 1 hour (2) a very long time.	0.01 - 0.5 h 0.5 - 3.0 h	5 min. 30 min.
Task switch issue factor	Factor that determines to which amount time needed for task switch will be taken into account when calculating the number of issues injected.	0.0 - 1.0	0.0
Fixing issue rate factor	Factor for the issue injection rate that determines how safe fixing is compared to creating new code.	0.2 - 1.1	0.3
Follow up issue spawn probability	Probability that an issue in a predecessor task leads to a follow up issue in a dependent task.	0.0 - 0.15	0.005

TABLE 2 Main model parameters (2/2)

Name/topic	Description	Sampling range (sec. 5.2)	Example value (sec. 5.1)
Review fix to task factor	Factor determining how time-consuming fixing an issue (without finding it) is in an issue fix task compared to a review remark.	0.9 – 2.0	1.1
Dependency graph constellation	Different types of task dependency structures, for example “NO_DEPENDENCIES” when there are no (relevant) dependencies between tasks of a story and “REALISTIC” for empirically observed structures.	NO_DER/ REALISTIC	REALISTIC [†]

study. The resulting intervals (and order of importance) are $\pm 7.5\%$ for quality, $\pm 10\%$ for efficiency, and $\pm 15\%$ for cycle time[‡].

3.2 | Iterative creation of the model

We used the model development process described by Ali and Petersen,²⁶ in addition to the guidelines from Law and Kelton²⁷ and Page and Kreuzer²⁸ (among others) to create the simulation model. Development of the model took about 4 months of calendar time.

The conceptual model was created based on review processes observed in practice. These observations mainly stem from the interviews we performed with software professionals from 19 companies, described in detail in Baum et al.⁵ and Baum et al.⁷ It was reviewed and checked for face validity by 2 experienced practitioners (the fourth and fifth author), 1 using pre-commit reviews and the other using post-commit reviews in his daily work. Both of these practitioners are lead software developers and have worked at several different companies. One has about 11 years of experience in industrial software development, the other about 19 years. The reviews of the conceptual model were conducted in form of a walk-through.²⁹ Further validation measures are described in section 6.

The model was implemented as a discrete event simulation model using the DESMO-J simulation framework.²⁸ For verification, we used a combination of several methods. Among those were software engineering best practices like unit tests and code reviews. Additionally, a group of 3 researchers checked parts of a simulation trace in detail by enacting it. Several iterations of global variance-based sensitivity analysis were performed using the method of Sobol'.³⁰ Parameters that were shown to be of little influence were set to fixed values. After several iterations for verification, we started to generate data to analyze the main effects and derive heuristics. Details for this sampling process are described in section 5.2.

3.3 | Data generation and analysis

The model is parametric, with input parameters for the details of the development context. These will be described in section 4 and Tables 1 and 2. For a given situation, the model is executed both using pre-commit and post-commit reviews, and the relative difference for the 3 output metrics is calculated. It is also executed without reviews,

to be able to exclude data points where any type of code review is not advisable. The execution is repeated with different random number seeds (using common random numbers²⁷ for variance reduction) to obtain a median difference and its confidence interval. Some more details on the data generation will be described in section 5.2.

To frame the heuristics, we used an iterative process based on exploratory data analysis and local sensitivity analysis. We formed working hypotheses, mainly by looking for correlations and other distinctive patterns in scatter plots of subsets of the simulation results. These hypotheses were then checked by 2 types of local sensitivity analysis³⁰: For a single data point, 1 factor at a time was increased and decreased, and 2 data points with opposing results were systematically interleaved. This process led to an increased understanding of the model's main effects but was also quite effective as a further method of verification.

4 | MODEL DESCRIPTION

The model's basic structure follows an agile Kanban-style³¹ development process, incorporating agile best practices and common industrial practices. A development team continuously pulls user stories from a pool of requirements, splits them into tasks, works on the tasks until all of them are finished and then delivers the results to the customers. After the implementation of a task is finished, the corresponding changes are reviewed. If issues are found during the review, they have to be fixed and another round of review follows, otherwise, the task is considered done. This basic life-cycle of a task is depicted in Figure 1[§]. Further quality assurance measures, like unit testing and static code analysis, are subsumed under “implementation.” We assume that some kind of “continuous delivery” process ensures that finished stories will be available to the customer shortly after they are finished, but do not model it in detail. Also out of scope is the prioritization process. Stories are simply created with random attributes when needed.

In section 1, we listed the effects that our interviewees believed to influence whether pre-commit or post-commit reviews are a better fit. All of these effects are considered in the model. Injection of “normal” and “blocker” issues, task switch overhead and conflicts on commit. The review variant is a special model parameter that will be used in some of the subprocesses described below. Figure 2 shows the classes

[‡]More details on the questionnaire are available online.⁹

[§]As advised in,²⁸ we use the UML to describe the model. The full diagrams are available online.⁹

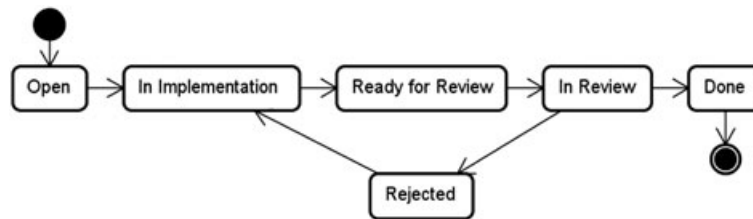
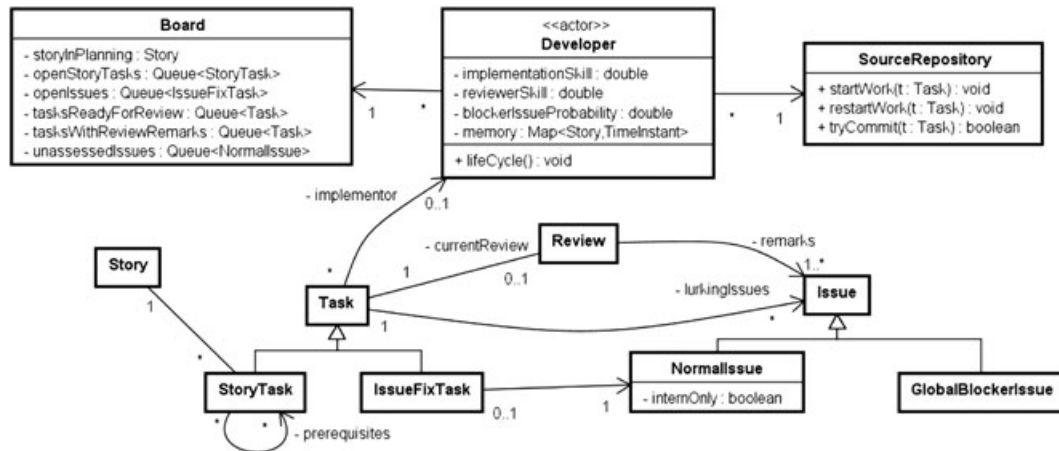
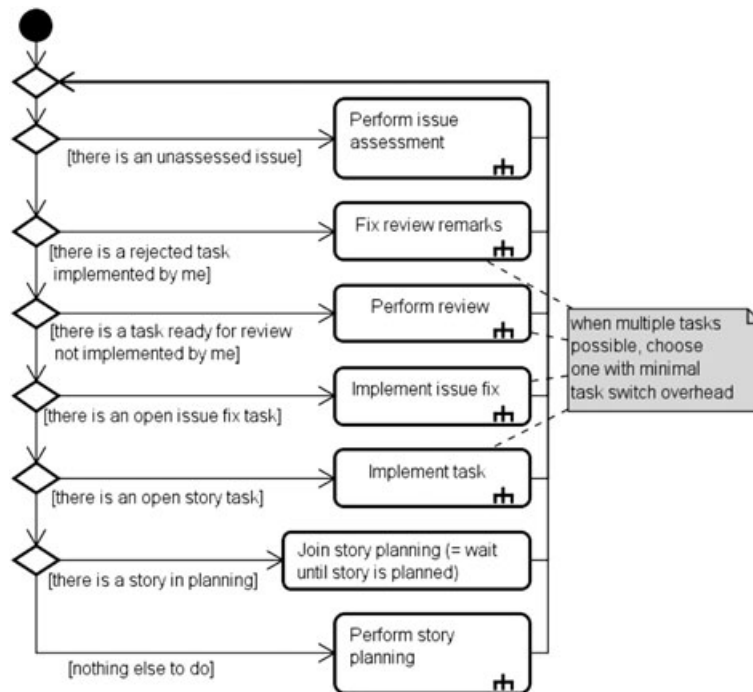
FIGURE 1 State diagram for *Tasks* (see also Figure 2)

FIGURE 2 Overview of important classes from the model

FIGURE 3 Activity diagram: main process followed by *Developer* (see also Section 4.1)

constituting the model. It contains 2 “active” parts: *Developers*[†] and *Issues*. We will first describe a *Developer*’s life-cycle in more detail and elaborate on *Issues* afterward. Not every detail of the model will be described as thoroughly as needed for detailed replication and scrutiny of our results. Therefore, the full model has been made available online.⁹

[†]In the rest of this section, class names will be written in *italic*.

4.1 | Details on the modeling of developers’ work

A *Developer* is an active process and follows the process model depicted in Figure 3. When looking for something to do, the *Developer* follows a strict priority order: When there is an *Issue* that just newly occurred, she has to look at it and decides what to do with it (“issue assessment”). When no such *Issues* exist, she checks whether a *Task* she implemented was rejected with review remarks and fixes these remarks. Next in

priority order comes the reviewing of *Tasks* implemented by others, then fixing of *IssueFixTasks*, then implementation of new *StoryTasks* and finally, when there is no other work to do, planning a new *Story* or joining someone else who is already planning.

The subprocess of implementing a task belonging to a story is shown in Figure 4. It starts with the *Developer* changing the *Task*'s state to "in implementation" and moving it to the corresponding column of the *Board*. She updates her local working copy from the source code repository, which registers the current simulation time for the later calculation of potential conflicts. The following 2 activities are shown separated, but could also be seen as interleaved. Depending on the time she last worked on the *Story*, she has to spend some "task switch overhead" to (re-)familiarize herself with it. Additionally, she performs the implementation work (including testing), which for the sake of this model mainly boils down to the creation of new *Issues*. The step after that depends on the review mode. In pre-commit review mode, the *Task*'s state is directly changed to "ready for review." In post-commit mode, she first has to commit her changes, during which there can be conflicts that have to be resolved.

When performing a review, the *Developer* first changes the *Task*'s state and has to cope with task switch overhead, too (see Figure 5). Then, the code is checked, which means that each *Issue* that is currently "lurking" in the *Task* is found with a certain developer-specific probability. If the developer found at least 1 *Issue*, she rejects the *Task* and notes the found *Issues* as review remarks. Otherwise, if in pre-commit mode, the *Task*'s accumulated changes will be committed to the main

development source tree, like described above. The *Task* is then considered "done," which could mean that the corresponding *Story* can be finished.

The process for fixing review remarks is similar to the process for the implementation of tasks. One of the main differences is that when fixing review remarks, the corresponding *Issue* will be marked as "fixed" on commit so that it cannot occur anymore (see Figure 6 for the *Issues*' states). Furthermore, the time needed for fixing depends on the sum of the single remarks' sizes and not the task's size.

4.2 | Details on the modeling of issues

The preceding section already alluded to the life-cycle of an *Issue*, which we will now describe in more detail. In our model, an *Issue* is very broadly defined as every true positive that can be remarked in a review.³² We distinguish 2 kinds of issues: *Normal Issues* stand for things like an incorrectly implemented algorithm (corresponding to the definition of fault in ISO/IEC/IEEE³³), as well as maintainability or performance problems. *Global Blocker Issues*, on the other hand, have been introduced to handle the special case when a *Developer* injects a problem that will block all other currently implementing *Developers* soon after it is committed. Depending on the context (eg, experienced developers vs first-time open source contributors or use of checks in hook scripts³⁴), the risk of such blocker issues can vary considerably. In contrast, when a *Normal Issue* is observed by a developer or customer, it is put in a queue with *Issues* that have to be assessed. During issue

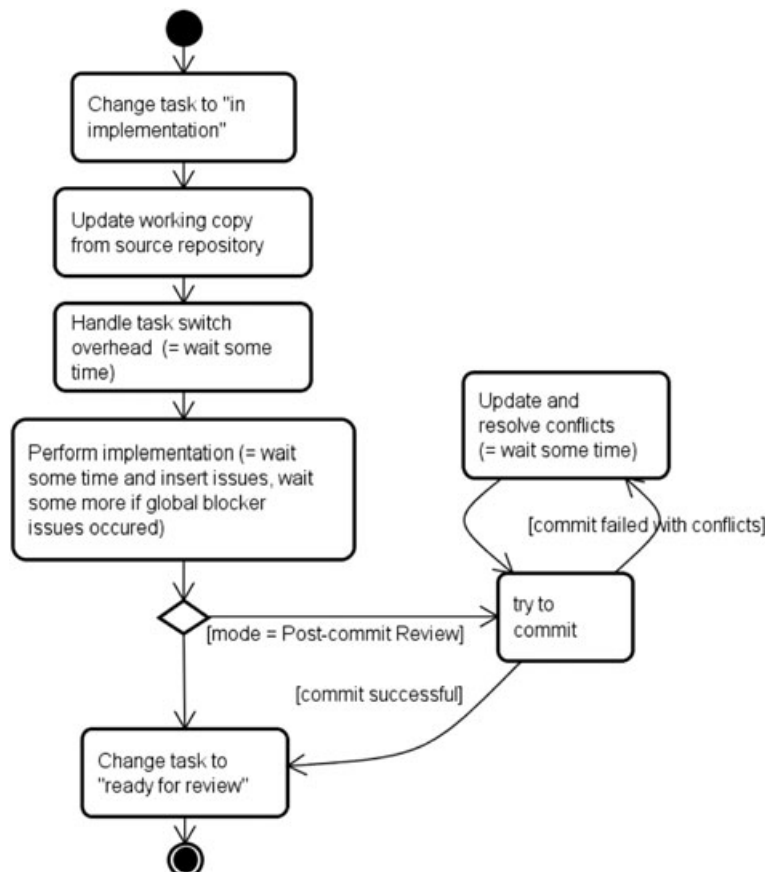


FIGURE 4 Activity diagram: details for the subprocess "Implement task" (from Figure 3)

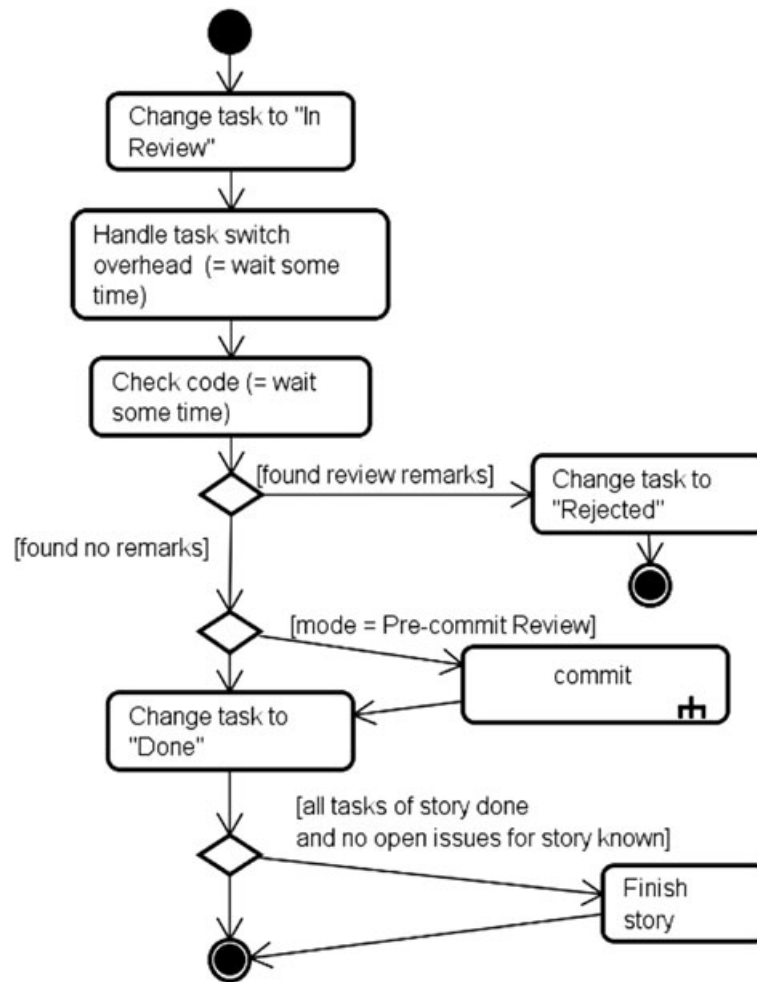


FIGURE 5 Activity diagram: details for the sub process “Perform review” (from Figure 3)

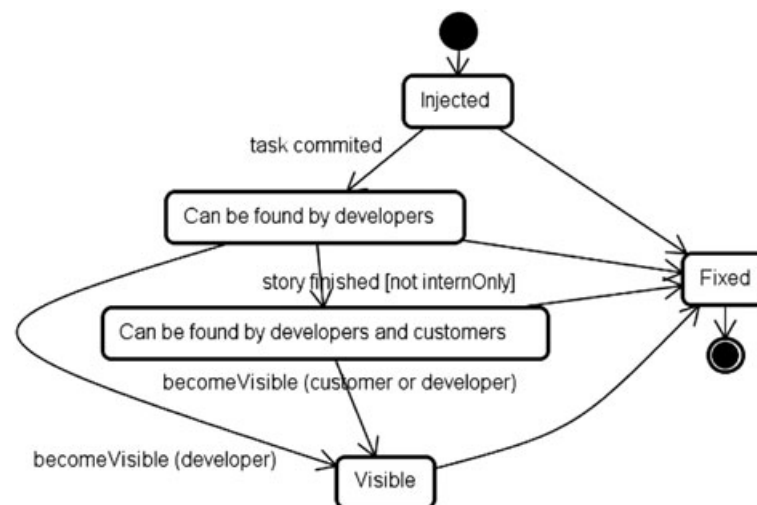


FIGURE 6 State diagram for *NormalIssues* (see also Figure 2)

assessment, a *Developer* determines which *Task* the *Issue* belongs to. Depending on the *Task*'s current state, the *Issue* can be fixed by some *Developer* who is currently at it anyway, or an *Issue Fix Task* has to be created. Conceptually, time lost due to distraction by an issue while working on another task is also modeled as part of the issue assessment time. Further details on issue assessment can be seen in Figure 7.

There are 2 ways a *Normal Issue* can become visible. It can be observed by a *Developer* after it has been committed to the main development source tree, and it can be observed by a customer after the story has been finished and delivered (see Figure 6). As soon as a fix for the *Issue* is committed, it cannot be observed anymore. Not all *Normal Issues* can be found by customers. A significant share of issues found

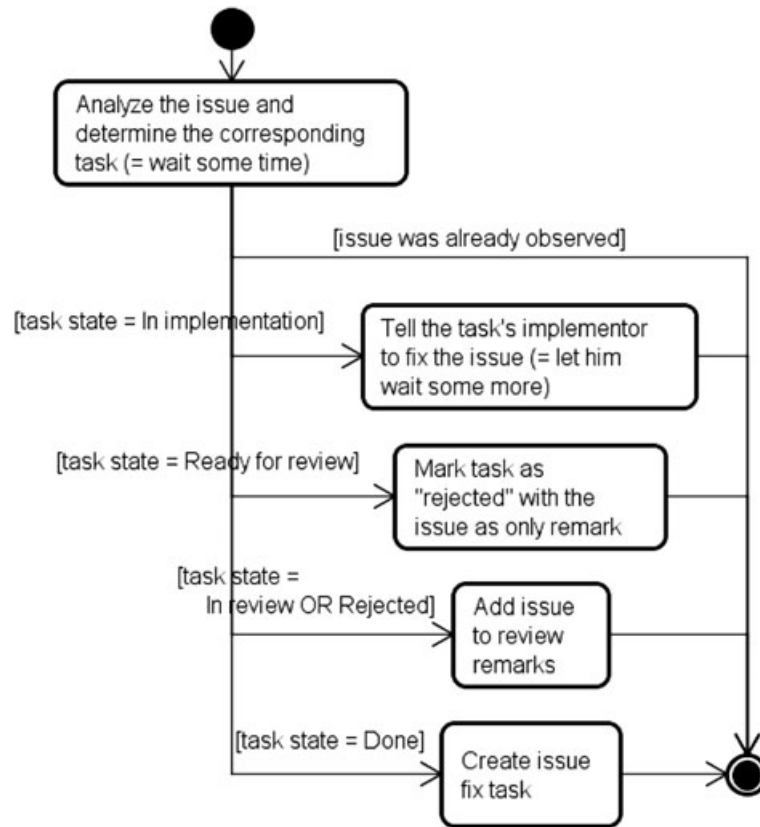


FIGURE 7 Activity diagram: details for the sub process “Perform issue assessment” (from Figure 3)

in code reviews are maintainability issues,³² and as such can only be observed by *Developers* (“internOnly”).

Issues can be injected every time a *Developer* implements a *StoryTask* or fixes a review remark or an *Issue Fix Task*. The number of *Issues* injected is calculated based on the *Developer*’s skill (measured in issues/hour for the sake of simplicity), and the time spent implementing (taken as a surrogate for the task’s complexity). An additional increase is due to follow-up issues¹:

$$\text{issue count} \approx \text{issue injection rate}_{\text{developer}} * \text{relevant time} + \text{follow up issue spawn probability} * \sum_{p \in \text{prerequisites}} |\text{lurking issues}_p| \quad (1)$$

When fixing issues, there is an additional “fixing issue rate factor” in the first part of the sum. It accounts for the possibly lower chance of introducing new issues when fixing old ones (due to fewer degrees of freedom). The relevant time depends on several parameters and on the type of the task. As an example, it is calculated for issue fix tasks as

$$\begin{aligned} \text{relevant time} := & \text{review remark fix duration} * \text{review fix to task factor} \\ & + \text{task switch overhead} * \text{task switch issue factor} \end{aligned} \quad (2)$$

The calculation of the task switch time is modeled with an exponential decay, inspired by the forgetting curve of Ebbinghaus.³⁵ Full details on its calculation are available in the model.⁹

¹Please refer to Tables 1 and 2 and Figure 2 for more details on the parameters and terms from the model used in the formulas. Like the rest of the model, the formulas were generated and validated with the iterative process described in Section 3.2

4.3 | Empirical triangulation of model parameters

As described in section 3, the main empirical foundation of our study is qualitative. In addition, we used quantitative empirical data in 3 ways: (1) to validate the shape of probability distributions for input parameters, (2) to determine realistic dependency structures, and (3) to cross-check our choice of input parameter ranges. The data were collected in retrospect from an industrial ticket system. This ticket system is used at a medium-sized software company (about 70 employees) that sells a complex software product to a limited number of large customers. It contains information on all software development tasks for this product for several years.

The distribution for the time between when an *Issue* could be observed and when it will be observed belongs to the first category: “determine the shape of probability distributions.” We sampled 15 user-found issues and manually determined the date they were injected and the date they were observed (ie, the bug ticket was created). The median difference was 207 days. The obtained distribution matched an exponential distribution fairly well. As it can also be argued on a theoretical basis that the time until an issue is found by a customer follows a (shifted) exponential distribution, we chose this distribution for the simulation.

To cross-check our choice of parameter ranges for the durations of task implementation and review, we also sampled data from the same ticket system. As this analysis was fully automated, we were able to take a larger sample. We excluded tasks with obviously erroneous durations, leaving us with 1896 tasks. The median time for implementation was about 4 working hours, the arithmetic mean about 9 working hours. For the review time, the median was 0.5 working hours, and the arithmetic

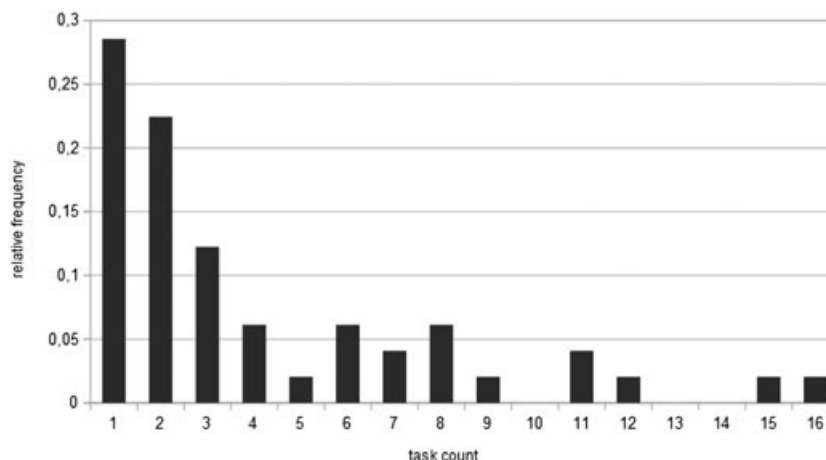


FIGURE 8 Relative frequency of different task counts per user story in the empirical sample

mean was 1 working hour. A day was counted as 8 working hours for the calculation. Both distributions are skewed to the right. The Pearson correlation coefficient of implementation and review time is (only) 0.18, so that we chose independent distributions for the implementation and review time. For distribution fitting, we compared log-normal, gamma, normal, and exponential distributions using R.³⁶ This calculation showed that a log-normal distribution is the best fit for all task durations.

As described in the previous section, new stories are created when needed in the simulation. During creation, the number of tasks and their dependency structure has to be determined. To gain realistic structures, we sampled 49 stories from the industrial ticket system. The dependency structure of the subtasks was then determined by a domain expert. The minimal number of tasks per story was 1, the maximal number in the sample was 16. Small numbers of tasks were much more frequent than larger numbers. The obtained empirical distribution of task counts can be seen in Figure 8. We encountered 25 different dependency graphs. To provide a little more insight into their structure, we determined the number of start tasks (tasks not dependent on anything) and end tasks (task on which no other tasks depends). In 14 of the graphs (15 of the stories), there are more end tasks than start tasks. In 4 cases, it is the other way around so that there is a tendency towards common prerequisite tasks followed by parallel work. In addition to this “REALISTIC” dependency structure, we analyzed 4 artificial structures: In “NO_SUBDIVISION” every story consists of only 1 task. In “NO_DEPENDENCIES,” the number of tasks per story is distributed like in “REALISTIC” but without any dependencies between the tasks. In “CHAINS,” the tasks are totally ordered so that their dependencies form a sequential structure. Every story contains at least 2 tasks, otherwise the distribution of small to large tasks is similar to “REALISTIC”. In the last studied structure, “DIAMONDS,” there is a single start task and a single end task. All tasks in between can be done in parallel. Consequently, every story consists of at least 3 tasks. During sensitivity analysis, it turned out that the exact dependency structure is not that relevant so that only “REALISTIC” and “NO_DEPENDENCIES” were studied in full detail. More information on the graph structure data can be found in the class *Dependency Graph Constellation* of Baum et al.⁹

Tables 1 and 2 summarize the main parameters of the model. A dagger (†) in the last column indicates that example values could be taken or estimated from empirical data.

5 | RESULTS

Our study's main target is to analyze a broad range of different contextual factors and to derive simple heuristics for the use in practice. These results will be shown in section 5.2, but beforehand we describe a specific data point in detail as a concrete example for the reader.

5.1 | Details for a specific data point

The example in this section is based on data from a specific team in the industry. The parameter values are shown in the right most column of Tables 1 and 2. The values marked with † were derived from the company's ticket system, and the rest is based on expert judgment. All these values have to be taken with a grain of salt. The ticket system data likely contains systematic biases, among others due to a tendency to forget to change a ticket's state or to pause working on a task without changing its state. The expert estimates are impeded by a lack of intuition for some of the parameters. As we are using this data point only as an example, most of these risks do not impair our main study results.

Figure 9 shows some output from a simulation run with these input values. It shows simulation time on the x-axis and the number of started and finished user stories as well as the number of issues observed by the customer on the y-axis. The sharp drop in the diagram marks the end of the warm-up period of 700 days. All counters were reset at that point. Only data collected in the 600 working days (about 3 years) after that were used for further analysis. At the very start of the simulation, the number of issues found by customers increases slowly due to the associated delays in the model. After that, all values show a roughly linear rise, as expected for a continuous Kanban process. It can be seen that for this data point, the warm-up period is by far large enough. We sized it this large to allow the same value to be used for data points which need a longer warm-up.

After 42 simulation runs with different random seeds, we obtained the results given in Table 3. For each output dimension, the table shows

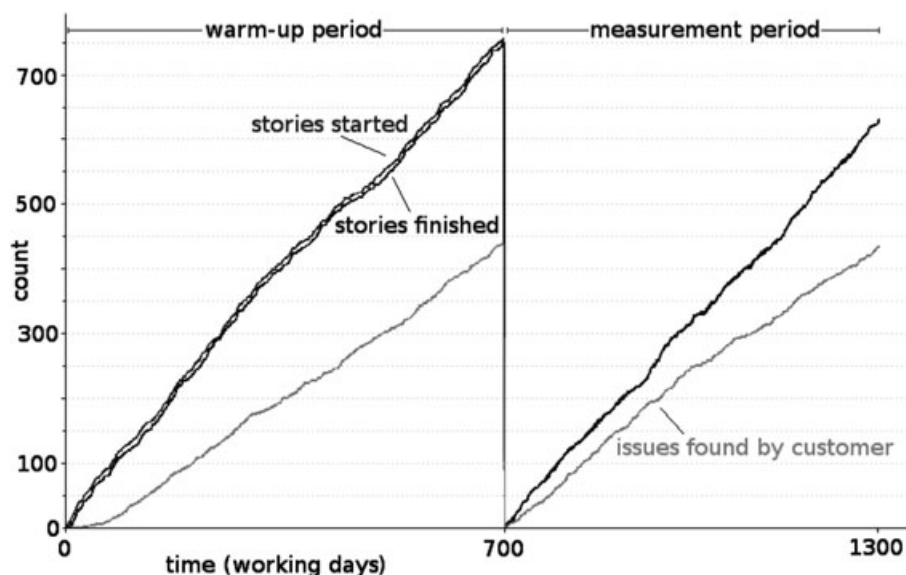


FIGURE 9 Example output from a simulation run: stories and issues over time

TABLE 3 Relative differences for the example data

	median	confidence interval	minimum	maximum
efficiency	0%	–1% to 1%	–3%	6%
quality	–2%	–5% to 1%	–11%	10%
cycle time	–12%	–15% to –11%	–22%	–6%

the median relative difference, the median's 99% confidence interval and the minimum and maximum relative difference obtained. Efficiency is measured in finished story points, quality in issues found by customers per story point, and cycle time by the mean cycle time of a user story. A positive relative difference means that post-commit reviews have the larger value. For efficiency, the larger value is better, for quality and cycle time, the smaller. All percentages have been rounded to the nearest integer for presentation. Those results show that the difference in efficiency and quality is very small for this data point. The difference in cycle time is much larger, but still not practically relevant, according to the limits for practical relevance gained in our survey (–15% to 15%).

5.2 | General results

To understand the influence of different contextual factors and to derive heuristics, we sampled a total of 289 230 random data points in our final analysis run⁹. This was done after several iterations of model verification, refinement, and parameter exclusion through global sensitivity analysis. To obtain a data point, each input value was sampled uniformly from the corresponding interval given in Tables 1 and 2. These intervals were estimated by the authors (including both practitioners), with a tendency to “if in doubt, enlarge.” For each sampled point, we repeatedly ran the simulation with different random number seeds at least 20 times. This process continued until a statistically significant

classification as 1 of “pre-better,” “post-better,” or “negligible (not practically relevant) difference” for each target attribute (number of story points finished, number of issues found by customers per story point, and mean story cycle time) was possible. Furthermore, we checked that the data point is not obviously unrealistic (more than 80% of the invested effort wasted on fixing etc), and that performing code review at all is actually beneficial (to err on the safe side, the number of finished story points without review had to be higher than 110% compared to the next best alternative to dismiss review). After excluding these cases, as well as those where statistical significance was not reached after 2000 repetitions, 50 901 data points remained. By far the largest number of exclusions were situations where almost no work was finished due to high error rates and ineffective fixing. Generating the data took about a week on a cluster of several high-end workstations. Based on these results and the combination of manual analysis and local sensitivity analysis described in section 3.3, we identified the main effects. They are described in the following.

5.2.1 | Analysis for quality

Regarding the number of issues found by customers per story point, there is 1 main effect that leads to a practically relevant difference between pre and post-commit reviews. Soon after a change is committed, other developers can spot (or “stumble upon”) issues. The time between this point and the delivery of the story is generally larger for post-commit than for pre-commit reviews, so that more issues are found outside of reviews. This effect is larger the higher the chance that developers will really do so, eg, they are really good at finding issues or there are many of them, and the effect is larger the higher the difference in time between post-commit and pre-commit reviews. The relative difference is also higher for lower review skills. That being said, the difference is not practically relevant for the vast majority of the sampled situations. The only effect we found that leads to a preference for pre-commit reviews in quality is an edge case. A high number of blocker issues leads to more review rounds for pre-commit reviews, and therefore increases total review effectiveness compared to post-commit reviews.

⁹The data set with simulation results is available online⁹

5.2.2 | Analysis for efficiency

For differences in efficiency, there are more effects interacting. As described for quality, post-commit reviews lead to more issues found by uninvolved developers. When review skills are low, this is a chance to find more issues while the author is still working at the task, which is beneficial for efficiency. With higher review skills, when most issues would be found in a review anyway, letting other developers find them is detrimental to efficiency. The relation between task switch overhead and review mode is double-edged, too. For small teams with task dependencies, task switch overhead is smaller for post-commit reviews, but for larger teams the task switch overhead for issues spotted by other developers grows and makes pre-commit better. Another relevant effect that is beneficial for pre-commit reviews stems from global blocker issues. The effect of blocker issues increases with a growing risk of injecting them, increasing time to recover from them and a growing number of affected developers. It also rises with the probability of finding blocker issues during a review.

5.2.3 | Analysis for cycle time

The largest difference between pre and post-commit reviews exists in cycle time. When there are dependencies between tasks so that 1 task has to be committed before another can be started, this leads to longer cycle times for pre-commit reviews. Figure 10 shows this quite well. The relative effect becomes larger when the time between initial implementation and the end of the review cycle grows in comparison to the total cycle time (e.g., through long review times or a large number of review rounds). This effect is so large that post-commit review has a smaller cycle time in most sampled cases with dependencies, whereas without dependencies a negligible difference in cycle times is prevalent. A second effect that affects cycle time is again the same as that responsible

for differences in quality. With post-commit review, more issues can be found by developers other than the reviewers. This usually takes more time than finding and fixing them as review remarks. When it does not, and review skills are low so that a significant number of issues will pop up after review, this can also lead to lower cycle times for post-commit review. The “global blocker” effect described for efficiency is relevant for cycle time, too.

5.2.4 | Resulting heuristics

We consolidated the following heuristics from our findings on the main effects and the simulation results:

For quality:

```
IF veryGoodAtSpottingIssues THEN Post
ELSE NegligibleDifference
```

For efficiency:

```
IF lowReviewSkill THEN
  IF veryGoodAtSpottingIssues
    THEN Post
  ELSE NegligibleDifference
ELSE
  IF globalIssuesAreAProblemSolvedByReviews
    OR veryGoodAtSpottingIssues
    THEN Pre
  ELSE IF smallTeam AND dependenciesRelevant
    AND shortIssueAssessment
    THEN Post
  ELSE NegligibleDifference
```

For cycle time:

```
IF lowReviewSkill OR NOT dependenciesRelevant
THEN NegligibleDifference
ELSE Post
```

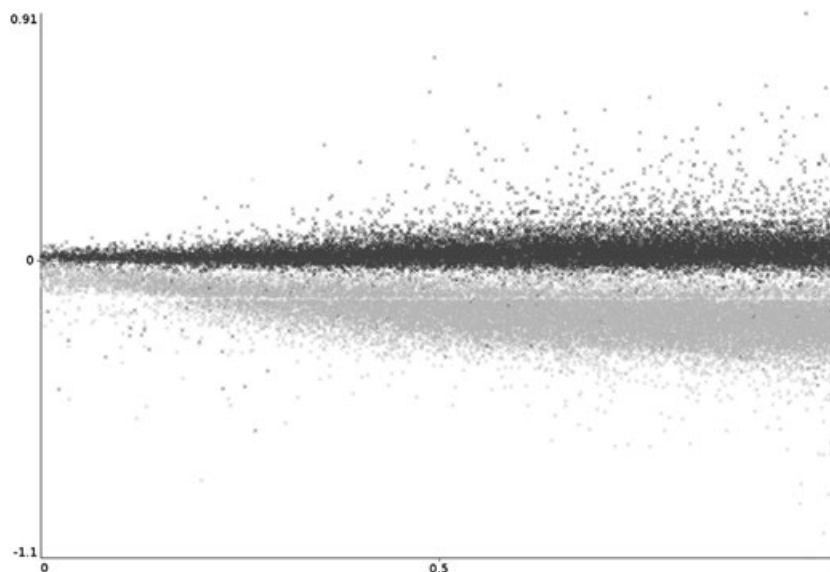


FIGURE 10 Scatter plot of review skill (x-axis), relative difference in cycle time (y-axis), and dependency graph constellation (color; light gray = REALISTIC, dark gray = NO_DEPENDENCIES)

TABLE 4 Confusion matrices

Heuristic for quality				Heuristic for efficiency				Heuristic for cycle time			
<i>pre</i>	0	4	271	<i>pre</i>	3201	12	1062	<i>pre</i>	0	39	531
<i>post</i>	0	10043	263	<i>post</i>	32	87	44	<i>post</i>	0	18738	397
<i>negl.</i>	0	2910	16693	<i>negl.</i>	9157	1564	35742	<i>negl.</i>	0	4143	27053
classified as →	<i>pre</i>	<i>post</i>	<i>negl.</i>	classified as →	<i>pre</i>	<i>post</i>	<i>negl.</i>	classified as →	<i>pre</i>	<i>post</i>	<i>negl.</i>

They use the following definitions, based on the parameters in Tables 1 and 2:

```

veryGoodAtSpottingIssues :=
  issue activation time mean for develop-
  ers < 75 days
lowReviewSkill := review skill < 25 %
globalIssuesAreAProblemSolvedByReviews :=
  (global blocker issue risk *
  global blocker issue suspend time *
  number of developers *
  review skill) > 1.5 person-hours
smallTeam := number of developers < 8
dependenciesRelevant :=
  dependency graph constella-
  tion != NO_DEPENDENCIES
shortIssueAssessment :=
  issue assessment time mean < 1.5 h

```

When formulating heuristics, the question is not if there will be inaccuracies, but rather which inaccuracies are least troublesome. We decided that, if in doubt, we value simplicity of the rules and recall for “pre-better” and “post-better” highest. The accuracy of the heuristics on the simulation results can be seen in the confusion matrices in Table 4, showing for each of the 3 result classes the number of data points classified by the heuristic as belonging to each class. For quality, we used only data points with at least 10 issues found by customers per year, because for lower values the “issues per story point” metric is dominated by differences in story points.

6 | LIMITATIONS

The 2 parts of our study, creation of the simulation model and derivation of heuristics, have their own threats to validity. To mitigate these, we mainly used techniques from Sargent RG³⁷ and de França BBN, Travassos GH³⁸ in addition to the guidelines referenced in section 3 and general software development best practices.

We implemented several measures to create a valid conceptual model. The general structure of the model is based on empirical evidence, gained from an in-depth analysis of the code review processes of several companies. Two software development practitioners performed an independent check for face validity. One of them uses pre-commit reviews in his daily work, the other post-commit reviews. In addition, the model was presented to and discussed with about 15 other software developers at a local Java User Group meeting. Consequently, we believe that the model has a firm qualitative empirical basis. Mainly due to missing data, the model's quantitative validation was much more limited. We estimated some parameter values and fitted distributions based on historical data from an industrial ticket system.

The simulation results of this data point were compared to the real historical outcomes. The results were encouraging, but their value is quite limited, as many of the model's parameters could be estimated only.

There are additional limitations regarding the quantitative empirical data we used. It is taken only from a single company, and it is based on the analysis of data collected for a different purpose. We believe that these limitations do not weaken our main study results, because we mainly used these values to cross-check our results. Where we did not, like for determining a realistic task dependency structure, we also ran the simulation model with alternative values. The ensuing sensitivity analysis showed the exact structures/values for these parameters to be of lesser importance.

Like every simulation model, our model is built for a specific purpose (comparison of pre and post-commit reviews) and contains simplifying assumptions, notably:

- A developer works on only 1 task at a time.
- Breaks and interruptions are not modeled, as are weekends and breaks between working days.
- There is a strict priority order of task types.
- The review round does not influence the time for a review, as well as the probability of finding an issue.
- A review is never canceled, even if there is a very high number of issues found.
- Review effects other than defect detection and spent effort (eg, better knowledge distribution or social problems) are not modeled. Neither are psychological effects.
- The model representation of the planning process is very simplistic (a developer starts planning alone, other developers join anytime, ...).
- While working on a task, a developer will not update his working copy.
- The process of developers finding issues while working on other tasks can be modeled with an exponential distribution.
- Soon after they are finished, stories will reach the customer (like continuous delivery with cherry picking) without further issues being injected (unlike cherry picking).
- The model contains no distinction between issues found by customers and issues found by system testing after development. That is “issues found by customers” really means “issues found by customers or by external QA” in the context of this study.
- The “topic” that determines if a change of topic leads to task switch overhead is the story the task belongs to.
- Dependencies between tasks only occur within a story.
- The same distribution is used for the issue assessment times of issues found by developers and issues found by customers.
- Normal issues can only be found once by a developer and once by a customer.

Regarding the verification of the computerized model, we also employed a combination of techniques. The model was developed iteratively, with a first throw-away prototype followed by incremental refinement. We used an established simulation framework²⁸ to reduce the chance of defects. We wrote automated test cases for expected reference behavior (mainly at extreme conditions), for example, that a lot of global issues lead to the superiority of pre-commit reviews and that a team doing reviews performs better than a team without reviews under normal circumstances. These test cases are accompanied by the use of assertions in the model's implementation. The automated tests were flanked by manual tests. We constantly used graphical depictions of the model behavior during analysis, and portions of a selected trace were acted upon in the form of a role play by 3 researchers. The extensive model execution during sensitivity analysis also helped to spot defects. In addition to testing, the computerized model's code was reviewed by 2 persons.

Our model is a probabilistic model. To draw valid conclusions, we interpret all results based on their median and its 99% confidence interval. Following the advice in Pawlikowski et al,³⁹ we repeated the simulation runs with differing random number seeds until the confidence interval was small enough to assign each data point to an output class with statistical significance, and we drew all random numbers from Mersenne twister pseudo-random number generators.

The heuristics we developed based on our model intentionally traded accuracy for simplicity. Nevertheless, we took some measures to ensure validity. When deciding whether to include a model parameter in the random sampling space, we used a policy of "if in doubt, leave it in," and fixed parameter values only if global sensitivity analysis showed them to be irrelevant. Our large random sample of 289 230 data points provides some protection against misinterpretation of random effects. We further used local sensitivity analysis to ensure that the correlations we observed in the output are really based on specific effects.

Regarding external validity, our model is based on a commonly used type of review process, but many real team's processes differ in the details. The main effects we identified for the heuristics can help to decide if the differences between a specific situation and our situation are relevant. For example, we believe that our results should be generalizable to Scrum teams by considering the generally increased story cycle time of this kind of process. The generalizability to teams using classic software development processes is probably rather limited. The external validity of our heuristics is limited by the fact that we do not know the empirical distributions of the input parameters for real development teams and therefore do not know for sure if an erroneous classification for a data point will affect thousands of real teams or no team at all. We only performed a systematic evaluation for teams up to 29 developers, and while many of the effects can be extrapolated, the heuristics would probably have to be adjusted for larger teams.

7 | DISCUSSION

In this section, we will elaborate on the implications of our results to practitioners as well as researchers, describe some of the lessons learned while conducting the study, and assess the potential for future work based on our results.

7.1 | Implications

Further boiling down the heuristics from section 5.2, we would summarize the main implication of our study to practitioners as: if you are currently doing code reviews and feel you have no problem with cycle time or developers being held back by issues that would be found in reviews, don't bother switching from post to pre or vice versa. If you don't have an existing process yet, use pre-commit reviews if your team is rather large, and cycle time doesn't matter much to you or you can arrange review so that no dependent task waits for the review to be finished. Use post-commit reviews if your team is rather small or you have dependencies between tasks that would otherwise increase cycle time^{††}. For pre-commit reviews in the form of pull requests, also keep in mind the benefits that were out of the scope of our study. Pull requests allow easier contribution to (open source) projects by outsiders,⁶ and they enforce more process discipline and are an easy way to keep unreviewed changes from being delivered to the customer.⁵ Also, note that what is described as post and pre-commit review in this study are actually extremes on a scale with multiple "mix processes" in between. Understanding of the main effects can help to find the right mix for a given situation.

For researchers, our study implies that there is no single "best" choice between pre and post-commit reviews so that both variants will remain in use in the foreseeable future. This has to be considered when developing code review tools and when investigating other parts of the review process.

7.2 | Methodological lessons learned

When looking at this research project in retrospect, we can identify things not only that worked out well but also things that didn't. In this section, we will describe some of them to help other researchers who want to use a similar methodology.

One of the main lessons learned is that it is quite easy to create *some* model, but that it is much more work to create a validated model that will be trusted. This is especially true in a situation like ours where empirical data for many of the input parameters was missing. It was helpful to have access to archival data that we could use to cross-check the parameter ranges and results. The discrete event simulation approach, the model representation in UML and its implementation in DESMO-J turned out to be well suited to discuss the model's validity with industrial practitioners. It was also very helpful that we could build on the results of a previous qualitative study to shape and delimit our research questions and the model.

A drawback of our reliance on face validity is that we had to include some model parameters "just in case". To mitigate this drawback, we used an iterative approach, refining the model and excluding parameters using sensitivity analysis. In retrospect, we probably should have used even shorter cycles in the beginning. In that way, we could have found more defects in the model's implementation early, before starting with large, time-consuming simulation runs.

^{††} These heuristics can be derived from the heuristics given in section 5.2 by assuming that (1) "veryGoodAtSpottingIssues" is false, (2) "globalIssuesAreAProblemSolvedByReviews" correlates with team size and is rare compared to differences in cycle time, and (3) "NegligibleDifference" can be treated as a "don't care" when optimizing the resulting boolean formulas.

In the beginning of our study, we planned to rely much more heavily on data mining to derive the heuristics than we finally did. Or, put differently, we learned that “understanding is key.” Using data mining, we generated rules with good values for precision and recall. But these rules were too complicated and did not help to explain the underlying effects. Therefore, we came to value mass analysis to find interesting data points, but manual analysis to understand the underlying mechanisms and derive heuristics.

7.3 | Relating the findings to the situational factors reference framework

Our model was mainly created inductively based on our qualitative findings. This approach, and the need to balance fidelity and complexity of the model, led to the use of the parameters described in Tables 1 and 2 in the model and later in the heuristics. Relating these parameters to the situational factors reference framework⁸ will ease the integration of our results into emerging models for process tailoring. Three general cases can be distinguished: (1) A subfactor can be directly mapped to a model parameter or is a generalization of a model parameter, (2) there is no corresponding subfactor for a parameter, and (3) there are intermediate variables linking the subfactor to the parameter. The factors “team size” (= “developer count”) and “programmer capability” (= “issue injection rate”) belong to the first category. In the second category, we could only identify the “review skill/capability” that is missing as a sub-factor of “skill.” The vast majority of factors and parameters belongs to the third category. For some of these, the relationship is quite obvious, for example “time to market” and “magnitude of potential loss” that influence the relative weighting of the target variables “quality,” “efficiency,” and “cycle time.” Many other factors are mediated by the software architecture and/or development process, for example for the parameters “dependency graph constellation,” “conflict probability,” and “global blocker issue risk.” The exact nature of these dependencies is out of the scope of the current article and material for future work. While there could exist a fourth category of factors that don't influence any of the parameters at all, we consider this to be highly unlikely, given the complex and interacting nature of software process and its context.⁴⁰

7.4 | Future work

As a study based mainly on simulation and with only limited empirical foundation, our results demand further triangulation. Furthermore, it was harder than expected to come up with reliable and simple heuristics mirroring the model's behavior. We had to trade simplicity for recall especially for cases classified as “negligible difference.” Using more elaborate techniques for analysis and rule description, future work could likely improve the heuristics. On the other hand, one of our main findings is that in many cases the choice between pre and post does not really matter in efficiency and effectiveness. In a way, this supports Porter et al's statement that “better defect detection techniques, not better process structures, are the key to improving inspection effectiveness”⁴¹, which was found in a quite different context. As a consequence, we believe that further research effort is probably better invested in other areas of code review research.

Software process simulation has often been described as a means for industry to help in process decisions. This has happened only to a very limited degree. We believe that this is due to the still quite high effort of creating a simulation model. Better building blocks for models, either in the form of reusable model components or in the form of formal, quantifiable theories, could reduce this effort. The problem of too large effort is amplified by the tendency in agile development to defer many process decisions to the development team, which normally has both not enough resources and not enough knowledge to use process simulation. The approach used in our study seems like a promising solution. Researchers, either in academia or in central departments of larger companies, use simulation to study a problem and to derive heuristics that can then be used by the singular development teams. Simulation used in this way is probably not the silver bullet for software engineering research but rather a valuable additional tool. Or in the words of Wernick and Hall, “Accept that there are severe limitations to the validity of results of purely simulation-based work, but treat this fact as an issue to be addressed rather than as a reason for dismissing them.”⁴²

8 | SUMMARY

This study compares 2 variants of change-based code review, pre-commit review and post-commit review, regarding differences in software development quality, efficiency, and cycle time. This is done with a parametric discrete event simulation model that was built based on observations of agile development processes. The model has been validated and verified in a number of ways, including checks for face validity by 1 lead software developer per variant. The simulation model is used to explore the main effects contributing to performance differences and to derive simple heuristics for the use in practice. Which variant is preferable depends on the context, for example if task dependencies exist and are relevant for reviews, how fast developers are at spotting issues and how large the team is. In many situations, there are no practically relevant differences. When there are, there is a tendency for post-commit review to be better regarding cycle time and quality and for pre-commit review regarding efficiency.

ACKNOWLEDGMENTS

The authors would like to thank Matthias Becker for his advice on simulation and Stephan Kiesling for help during trace checking. Further thanks go to the respondents to the questionnaire on the meaning of “practical relevance”.

REFERENCES

1. Fagan M. Design and code inspections to reduce errors in program development. *IBM Syst J*. 1976;15(3):182–211.
2. Gilb T, Graham D. *Software Inspection*, Addison-Wesley: Wokingham, UK; 1993.
3. Bernhart M, Grechenig T. On the understanding of programs with continuous code reviews. *Program Comprehension (ICPC), 2013 IEEE 21st International Conference on*, IEEE: San Francisco, CA, USA; 2013: 192–198.
4. Rigby PC, Bird C. Convergent contemporary software peer review practices. *Proceedings of the 2013 9th Joint Meeting on Foundations of Software Engineering*, ACM: Saint Petersburg, Russia; 2013:202–212.

5. Baum T, Liskin O, Niklas K, Schneider K. A faceted classification scheme for change-based industrial code review processes. *Software Quality, Reliability and Security (QRS), 2016 IEEE International Conference on*, IEEE: Vienna, Austria; 2016. <https://doi.org/10.1109/QRS.2016.19>.
6. Gousios G, Pinzger M, Deursen A. An exploratory study of the pull-based software development model. *Proceedings of the 36th International Conference on Software Engineering*, ACM: Hyderabad, India; 2014:345–355.
7. Baum T, Liskin O, Niklas K, Schneider K. Factors influencing code review processes in industry. *Proceedings of the ACM SIGSOFT 24th International Symposium on the Foundations of Software Engineering*, ACM: Seattle, WA, USA; 2016. <https://doi.org/10.1145/2950290.2950323>.
8. Clarke P, O'Connor RV. The situational factors that affect the software development process: Towards a comprehensive reference framework. *Inf Software Technol*. 2012;54(5):433–447.
9. Baum T, Kortum F. Simulation results and source code for pre/post-commit comparison paper. <https://github.com/FG-SE/PrePostReviewProcessSimulation/releases/tag/v1.0>. Accessed on 2017-03-12
10. Baum T, Kortum F, Schneider K, Brack A, Schauder J. Comparing pre-commit reviews and post-commit reviews using process simulation. *Software and System Process (ICSSP), 2016 International Conference on*, Austin, TX, USA; 2016.
11. Rigby PC. Understanding open source software peer review: Review processes, parameters and statistical models, and underlying behaviours and mechanisms. *PhD Thesis*, University of Victoria, 2011.
12. Madachy RJ. System dynamics modeling of an inspection-based process. *Software Engineering, 1996., Proceedings of the 18th International Conference on*, IEEE: Berlin, Germany; 1996:376–386.
13. Tvedt JD, Collofello J. Evaluating the effectiveness of process improvements on software development cycle time via system dynamics modelling. *Computer Software and Applications Conference, 1995. COMPSAC 95. Proceedings., Nineteenth Annual International*, IEEE: Dallas, TX, USA; 1995:318–325.
14. Neu H, Hanne T, Münch J, Nickel S, Wirsén A. Creating a code inspection model for simulation-based decision support. *ProSim*. 2003;3:3–4.
15. Rus I, Halling M, Biffi S. Supporting decision-making in software engineering with process simulation and empirical studies. *Int J Software Eng Knowledge Eng*. 2003;13(05):531–545.
16. Münch J, Armbrust O. Using empirical knowledge from replicated experiments for software process simulation: A practical example. *Empirical Software Engineering, 2003. ISESE 2003. Proceedings. 2003 International Symposium on*, IEEE: Rome, Italy; 2003:18–27.
17. Wakeland WW, Martin RH, Raffo D. Using design of experiments, sensitivity analysis, and hybrid simulation to evaluate changes to a software development process: a case study. *Software Process: Improv Pract*. 2004;9(2):107–119.
18. Turnu I, Melis M, Cau A, Setzu A, Concas G, Mannaro K. Modeling and simulation of open source development using an agile practice. *J Syst Archit*. 2006;52(11):610–618.
19. Melis M, Turnu I, Cau A, Concas G. Evaluating the impact of test-first programming and pair programming through software process simulation. *Software Process: Improv Pract*. 2006;11(4):345–360.
20. Anderson DJ, Concas G, Lunesu MI, Marchesi M, Zhang H. A comparative study of scrum and kanban approaches on a real case study using simulation. *Agile Processes in Software Engineering and Extreme Programming*, Springer: Malmö, Sweden; 2012:123–137.
21. Zhang H, Kitchenham B, Pfahl D. Software process simulation modeling: an extended systematic review. *New Modeling Concepts for Today's Software Processes*, Springer: Paderborn, Germany; 2010:309–320.
22. Ali NB, Petersen K, Wohlin C. A systematic literature review on the industrial use of software process simulation. *J Syst Software*. 2014;97:65–85.
23. Clarke P, O'Connor RV, Leavy B, Yilmaz M. Exploring the relationship between software process adaptive capability and organisational performance. *IEEE Trans Software Eng*. 2015;41(12):1169–1183.
24. MacCormack A, Verganti R. Managing the sources of uncertainty: Matching process and context in software development. *J Prod Innov Manage*. 2003;20(3):217–232.
25. Jeners S, O'Connor R, Clarke P, Lichter H, Lepmets M, Buglione L. Harnessing software development contexts to inform software process selection decisions. *Software Qual Prof*. 2013;16(1):35–36.
26. Ali NB, Petersen K. A consolidated process for software process simulation: State of the art and industry experience. *Software Engineering and Advanced Applications (SEAA), 2012 38th EUROMICRO Conference on*, IEEE: Cesme, Turkey; 2012:327–336.
27. Law AM, Kelton WD. *Simulation Modeling and Analysis*, McGraw-Hill: Boston, MA, USA; 2000.
28. Page B, Kreuzer W. *The Java Simulation Handbook – Simulating Discrete Event Systems with UML and Java*, Shaker: Aachen, Germany; 2005.
29. IEEE. IEEE Standard for Software Reviews and Audits. IEEE1028-2008; 2008.
30. Saltelli A, Chan K, Scott EM. *Sensitivity Analysis*, John Wiley & Sons Inc.: Chichester, UK; 2000.
31. Anderson D. *Kanban: Successful Evolutionary Change for Your Technology Business*, Blue Hole Press: Sequim, WA, USA; 2010.
32. Mantyla MV, Lassenius C. What types of defects are really discovered in code reviews? *Software Eng IEEE Trans*. 2009;35(3):430–448.
33. ISO/IEC/IEEE. Systems and software engineering vocabulary. ISO/IEC/IEEE 24765:2010; 2010.
34. Baum T. Leveraging pre-commit hooks for context-sensitive checklists: a case study. *Fachtagung des GI-Fachbereichs Softwaretechnik, Software Engineering (SE 2015), Dresden, Germany; 2015:219–222*.
35. Madachy R. *Software Process Dynamics*, Wiley: Hoboken, NJ, USA; 2007.
36. Core Team R. R: A Language and Environment for Statistical Computing. Vienna, Austria: R Foundation for Statistical Computing; 2014. <https://www.R-project.org>.
37. Sargent RG. Verification and validation of simulation models. *Proceedings of the 37th Conference on Winter Simulation, Winter Simulation Conference*, Orlando, FL, USA; 2005:130–143.
38. de Franca BBN, Travassos GH. Simulation based studies in software engineering: A matter of validity. *CLEI Electron J*. 2015;18(1):5–5.
39. Pawlikowski K, Jeong HD, Lee JS. On credibility of simulation studies of telecommunication networks. *Commun Mag IEEE*. 2002;40(1):132–139.
40. Clarke P, O'Connor RV, Leavy B. A complexity theory viewpoint on the software development process and situational context. *Proceedings of the International Conference on Software and Systems Process*, ACM: Austin, TX, USA; 2016:86–90.
41. Porter A, Siy H, Mockus A, Votta L. Understanding the sources of variation in software inspections. *ACM Trans Software Eng Method (TOSEM)*. 1998;7(1):41–79.
42. Wernick P, Hall T. Getting the best out of software process simulation and empirical research in software engineering. *Proceedings of the Second International Workshop on Realising Evidence-Based Software Engineering*, IEEE Computer Society: Minneapolis, MN, USA; 2007:3.

How to cite this article: Baum T, Kortum F, Schneider K, Brack A, Schauder J. Comparing pre-commit reviews and post-commit reviews using process simulation. *J Softw Evol Proc*. 2017;29:e1865. <https://doi.org/10.1002/smr.1865>